

LeetCode Placement Companion

Complete 70-Problem Interview Reference Manual (Java)

1. Arrays

| 1. Two Sum

PROBLEM DEFINITION

Given an array of integers `nums` and an integer `target`, return *indices* of the two numbers such that they add up to `target`. You may assume that each input would have ***exactly* one solution**, and you may not use the *same* element twice.

PRIMARY TEST CASES

```
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
```

APPROACH & KEY INSIGHTS

Use a HashMap to store the value and its index as you iterate through the array. For each element, check if `target - nums[i]` already exists in the map. This achieves an $O(n)$ time complexity compared to the brute-force $O(n^2)$ approach.

PERFECT JAVA SOLUTION

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (map.containsKey(complement)) {
                return new int[] { map.get(complement), i };
            }
            map.put(nums[i], i);
        }
        return new int[] {};
    }
}
```

2. Best Time to Buy and Sell Stock

PROBLEM DEFINITION

You are given an array `prices` where `prices[i]` is the price of a given stock on the `i-th` day. You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock. Return **the maximum profit you can achieve from this transaction**. If you cannot achieve any profit, return `0`.

PRIMARY TEST CASES

Input: prices = [7,1,5,3,6,4]

Output: 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.

APPROACH & KEY INSIGHTS

Track the minimum price seen so far as you iterate through the array. At each step, calculate the potential profit if sold today ($\text{prices}[i] - \text{minPrice}$) and update the maximum profit accordingly.

PERFECT JAVA SOLUTION

```
class Solution {
    public int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE;
        int maxProfit = 0;
        for (int price : prices) {
            if (price < minPrice) {
                minPrice = price;
            } else if (price - minPrice > maxProfit) {
                maxProfit = price - minPrice;
            }
        }
        return maxProfit;
    }
}
```

3. Product of Array Except Self

PROBLEM DEFINITION

Given an integer array `nums`, return *an array* `answer` *such that* `answer[i]` *is equal to the product of all the elements of* `nums` *except* `nums[i]`. The product of any prefix or suffix of `nums` is guaranteed to fit in a **32-bit** integer. You must write an algorithm that runs in $O(n)$ time and without using the division operation.

PRIMARY TEST CASES

Input: `nums = [1,2,3,4]`
Output: `[24,12,8,6]`

APPROACH & KEY INSIGHTS

Construct the result array by utilizing two passes. In the first pass, compute the prefix product for each element. In the second pass, compute the suffix product on the fly and multiply it into the corresponding position.

PERFECT JAVA SOLUTION

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];
        ans[0] = 1;
        for (int i = 1; i < n; i++) {
            ans[i] = ans[i - 1] * nums[i - 1];
        }
        int right = 1;
        for (int i = n - 1; i >= 0; i--) {
            ans[i] *= right;
            right *= nums[i];
        }
        return ans;
    }
}
```

4. Maximum Subarray

PROBLEM DEFINITION

Given an integer array `nums`, find the subarray with the largest sum, and return *its sum*.

PRIMARY TEST CASES

Input: nums = [-2,1,-3,4,-1,2,1,-5,4]

Output: 6

Explanation: The subarray [4,-1,2,1] has the largest sum = 6.

APPROACH & KEY INSIGHTS

Use Kadane's Algorithm. Traverse the array while keeping track of the current subarray sum (`currentSum`). If `currentSum` becomes negative, reset it to 0 (or start fresh from the current element). Track the global maximum sum discovered.

PERFECT JAVA SOLUTION

```
class Solution {
    public int maxSubArray(int[] nums) {
        int maxSoFar = nums[0];
        int currentMax = nums[0];
        for (int i = 1; i < nums.length; i++) {
            currentMax = Math.max(nums[i], currentMax + nums[i]);
            maxSoFar = Math.max(maxSoFar, currentMax);
        }
        return maxSoFar;
    }
}
```

5. Merge Intervals

PROBLEM DEFINITION

Given an array of `intervals` where `intervals[i] = [start\_i, end\_i]`, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input*.

PRIMARY TEST CASES

Input: intervals = [[1,3],[2,6],[8,10],[15,18]]

Output: [[1,6],[8,10],[15,18]]

Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].

APPROACH & KEY INSIGHTS

Sort the intervals by their start times. Iterate through the sorted list, maintaining the current active interval. If the start of the next interval is less than or equal to the end of the current interval, merge them by updating the end time.

PERFECT JAVA SOLUTION

```
class Solution {
    public int[][] merge(int[][] intervals) {
        if (intervals.length <= 1) return intervals;
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
        List result = new ArrayList<>();
        int[] currentInterval = intervals[0];
        result.add(currentInterval);
        for (int[] interval : intervals) {
            if (interval[0] <= currentInterval[1]) {
                currentInterval[1] = Math.max(currentInterval[1], interval[1]);
            } else {
                currentInterval = interval;
                result.add(currentInterval);
            }
        }
        return result.toArray(new int[result.size()][]);
    }
}
```

6. Rotate Array

PROBLEM DEFINITION

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

PRIMARY TEST CASES

Input: `nums = [1,2,3,4,5,6,7], k = 3`
Output: `[5,6,7,1,2,3,4]`

APPROACH & KEY INSIGHTS

First normalize `k = k % nums.length`. Then reverse the entire array. Next, reverse the first `k` elements, and finally reverse the remaining `n - k` elements. This shifts the elements in-place with $O(1)$ extra space.

PERFECT JAVA SOLUTION

```
class Solution {
    public void rotate(int[] nums, int k) {
        k %= nums.length;
        reverse(nums, 0, nums.length - 1);
        reverse(nums, 0, k - 1);
        reverse(nums, k, nums.length - 1);
    }
    private void reverse(int[] nums, int start, int end) {
        while (start < end) {
            int temp = nums[start];
            nums[start] = nums[end];
            nums[end] = temp;
            start++;
            end--;
        }
    }
}
```

7. Set Matrix Zeroes

PROBLEM DEFINITION

Given an $m \times n$ integer matrix `matrix`, if an element is `0`, set its entire row and column to `0`'s. You must do it **in-place**.

PRIMARY TEST CASES

```
Input: matrix = [[1,1,1],[1,0,1],[1,1,1]]
Output: [[1,0,1],[0,0,0],[1,0,1]]
```

APPROACH & KEY INSIGHTS

Use the first row and first column of the matrix itself as flags to store whether that row/column should be zeroed out. Use two extra boolean flags to remember if the first row and column themselves originally contained zeroes.

PERFECT JAVA SOLUTION

```
class Solution {
    public void setZeroes(int[][] matrix) {
        boolean rowZero = false, colZero = false;
        int m = matrix.length, n = matrix[0].length;
        for (int i = 0; i < m; i++) if (matrix[i][0] == 0) colZero = true;
        for (int j = 0; j < n; j++) if (matrix[0][j] == 0) rowZero = true;
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (matrix[i][j] == 0) {
                    matrix[i][0] = 0;
                    matrix[0][j] = 0;
                }
            }
        }
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                    matrix[i][j] = 0;
                }
            }
        }
        if (colZero) for (int i = 0; i < m; i++) matrix[i][0] = 0;
        if (rowZero) for (int j = 0; j < n; j++) matrix[0][j] = 0;
    }
}
```


2. Strings

| 1. Valid Anagram

PROBLEM DEFINITION

Given two strings `s` and `t`, return `true` if* `t` *is an anagram of* `s`, and* `false` *otherwise*.

PRIMARY TEST CASES

```
Input: s = "anagram", t = "nagaram"
Output: true
```

APPROACH & KEY INSIGHTS

Create a frequency array of size 26. Increment counts for characters in `s` and decrement for `t`. If all positions are zero at the end, the strings are anagrams.

PERFECT JAVA SOLUTION

```
class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        int[] counts = new int[26];
        for (int i = 0; i < s.length(); i++) {
            counts[s.charAt(i) - 'a']++;
            counts[t.charAt(i) - 'a']--;
        }
        for (int count : counts) if (count != 0) return false;
        return true;
    }
}
```

2. Longest Common Prefix

PROBLEM DEFINITION

Write a function to find the longest common prefix string amongst an array of strings. If there is no common prefix, return an empty string `""`.

PRIMARY TEST CASES

```
Input: strs = ["flower", "flow", "flight"]
Output: "fl"
```

APPROACH & KEY INSIGHTS

Initialize the prefix as the first string. Iteratively trim the prefix characters from the end until it matches the beginning of the next string using `indexOf`.

PERFECT JAVA SOLUTION

```
class Solution {
    public String longestCommonPrefix(String[] strs) {
        if (strs == null || strs.length == 0) return "";
        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (strs[i].indexOf(prefix) != 0) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) return "";
            }
        }
        return prefix;
    }
}
```

3. Valid Palindrome

PROBLEM DEFINITION

A phrase is a **palindrome** if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers. Given a string `s`, return `true` if it is a palindrome, or `false` otherwise.

PRIMARY TEST CASES

```
Input: s = "A man, a plan, a canal: Panama"
Output: true
```

APPROACH & KEY INSIGHTS

Use a two-pointer approach, moving inwards from both ends. Skip any non-alphanumeric characters using `Character.isLetterOrDigit()`, checking character match at each step ignoring case.

PERFECT JAVA SOLUTION

```
class Solution {
    public boolean isPalindrome(String s) {
        int i = 0, j = s.length() - 1;
        while (i < j) {
            while (i < j && !Character.isLetterOrDigit(s.charAt(i))) i++;
            while (i < j && !Character.isLetterOrDigit(s.charAt(j))) j--;
            if (Character.toLowerCase(s.charAt(i)) != Character.toLowerCase(s.charAt(j)))
                return false;
            i++;
            j--;
        }
        return true;
    }
}
```

| 4. Group Anagrams

PROBLEM DEFINITION

Given an array of strings `strs`, group the **anagrams** together. You can return the answer in **any order**.

PRIMARY TEST CASES

```
Input: strs = ["eat","tea","tan","ate","nat","bat"]
Output: [["bat"],["nat","tan"],["ate","eat","tea"]]
```

APPROACH & KEY INSIGHTS

Sort each string alphabetically to use it as a uniform key for a HashMap. Group strings matching the same key into a collection list.

PERFECT JAVA SOLUTION

```
class Solution {
    public List<String> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strs) {
            char[] chars = s.toCharArray();
            Arrays.sort(chars);
            String key = String.valueOf(chars);
            if (!map.containsKey(key)) map.put(key, new ArrayList<>());
            map.get(key).add(s);
        }
        return new ArrayList<>(map.values());
    }
}
```

5. Longest Substring Without Repeating Characters

PROBLEM DEFINITION

Given a string `s`, find the length of the **longest substring** without repeating characters.

PRIMARY TEST CASES

Input: s = "abcabcbb"

Output: 3

Explanation: The answer is "abc", with the length of 3.

APPROACH & KEY INSIGHTS

Use a sliding window with a HashSet or HashMap. Advance the right pointer to add new elements. If a duplicate is encountered, slide the left pointer forward until the window becomes distinct.

PERFECT JAVA SOLUTION

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        Set set = new HashSet<>();
        int left = 0, maxLen = 0;
        for (int right = 0; right < s.length(); right++) {
            while (set.contains(s.charAt(right))) {
                set.remove(s.charAt(left));
                left++;
            }
            set.add(s.charAt(right));
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
}
```

6. String to Integer (atoi)

PROBLEM DEFINITION

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer (similar to C/C++'s `atoi` function). Handle whitespaces, leading signs, and integer overflows appropriately.

PRIMARY TEST CASES

Input: `s = " -42"`
Output: `-42`

APPROACH & KEY INSIGHTS

Discard leading whitespaces. Read optional '+' or '-'. Process numeric digits while protecting against 32-bit overflow thresholds via simple comparisons before adding.

PERFECT JAVA SOLUTION

```
class Solution {
    public int myAtoi(String s) {
        if (s == null || s.isEmpty()) return 0;
        int i = 0, sign = 1, n = s.length();
        long res = 0;
        while (i < n && s.charAt(i) == ' ') i++;
        if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
            sign = (s.charAt(i) == '-') ? -1 : 1;
            i++;
        }
        while (i < n && Character.isDigit(s.charAt(i))) {
            res = res * 10 + (s.charAt(i) - '0');
            if (sign * res > Integer.MAX_VALUE) return Integer.MAX_VALUE;
            if (sign * res < Integer.MIN_VALUE) return Integer.MIN_VALUE;
            i++;
        }
        return (int) (sign * res);
    }
}
```

7. Minimum Window Substring

PROBLEM DEFINITION

Given two strings `s` and `t` of lengths `m` and `n` respectively, return *the minimum window substring of `s` such that every character in `t` (including duplicates) is included in the window*.

PRIMARY TEST CASES

```
Input: s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"
```

APPROACH & KEY INSIGHTS

Use a sliding window map framework tracking character frequency balances. Expand the window right until valid, then compress left dynamically while checking target constraints.

PERFECT JAVA SOLUTION

```
class Solution {
    public String minWindow(String s, String t) {
        if (s.length() < t.length()) return "";
        int[] tCount = new int[128];
        for (char c : t.toCharArray()) tCount[c]++;
        int left = 0, right = 0, minLeft = 0, minLen = Integer.MAX_VALUE, count =
t.length();
        while (right < s.length()) {
            if (tCount[s.charAt(right)] > 0) count--;
            tCount[s.charAt(right)]--;
            right++;
            while (count == 0) {
                if (right - left < minLen) {
                    minLen = right - left;
                    minLeft = left;
                }
                tCount[s.charAt(left)]++;
                if (tCount[s.charAt(left)] > 0) count++;
                left++;
            }
        }
        return minLen == Integer.MAX_VALUE ? "" : s.substring(minLeft, minLeft + minLen);
    }
}
```

3. Linked List

1. Reverse Linked List

PROBLEM DEFINITION

Given the `head` of a singly linked list, reverse the list, and return *the reversed list*.

PRIMARY TEST CASES

Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]

APPROACH & KEY INSIGHTS

Maintain three continuous node reference modifications in loop cycles via `prev`, `curr`, and `next` pointer transformations.

PERFECT JAVA SOLUTION

```
class Solution {  
    public ListNode reverseList(ListNode head) {  
        ListNode prev = null, curr = head;  
        while (curr != null) {  
            ListNode next = curr.next;  
            curr.next = prev;  
            prev = curr;  
            curr = next;  
        }  
        return prev;  
    }  
}
```


2. Middle of the Linked List

PROBLEM DEFINITION

Given the `head` of a singly linked list, return *the middle node of the linked list*. If there are two middle nodes, return **the second middle** node.

PRIMARY TEST CASES

Input: head = [1,2,3,4,5,6]
Output: Node 4

APPROACH & KEY INSIGHTS

Implement Floyd's Tortoise and Hare approach. Advance `slow` pointer by one node and `fast` pointer by two nodes per step.

PERFECT JAVA SOLUTION

```
class Solution {  
    public ListNode middleNode(ListNode head) {  
        ListNode slow = head, fast = head;  
        while (fast != null && fast.next != null) {  
            slow = slow.next;  
            fast = fast.next.next;  
        }  
        return slow;  
    }  
}
```

3. Merge Two Sorted Lists

PROBLEM DEFINITION

You are given the heads of two sorted linked lists `list1` and `list2`. Merge the two lists in a one sorted list. The list should be made by splicing together the nodes of the first two lists.

PRIMARY TEST CASES

```
Input: list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]
```

APPROACH & KEY INSIGHTS

Use a dummy node to seamlessly hook growing chain comparisons until one stream empties, then anchor the trailing sorted remains.

PERFECT JAVA SOLUTION

```
class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;
        while (list1 != null && list2 != null) {
            if (list1.val <= list2.val) {
                curr.next = list1;
                list1 = list1.next;
            } else {
                curr.next = list2;
                list2 = list2.next;
            }
            curr = curr.next;
        }
        curr.next = (list1 != null) ? list1 : list2;
        return dummy.next;
    }
}
```

4. Linked List Cycle

PROBLEM DEFINITION

Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

PRIMARY TEST CASES

Input: head = [3,2,0,-4], pos = 1
Output: true

APPROACH & KEY INSIGHTS

Deploy Fast and Slow runner pointers. If a cycle exists, the fast runner will eventually overlap and catch up to the slow runner.

PERFECT JAVA SOLUTION

```
class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
}
```

5. Remove Nth Node From End of List

PROBLEM DEFINITION

Given the `head` of a linked list, remove the `n-th` node from the end of the list and return its head.

PRIMARY TEST CASES

Input: head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]

APPROACH & KEY INSIGHTS

Advance a fast reference pointer `n` times ahead first. Then run fast and slow references uniformly until fast touches the boundary edge.

PERFECT JAVA SOLUTION

```
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode first = dummy, second = dummy;
        for (int i = 0; i <= n; i++) first = first.next;
        while (first != null) {
            first = first.next;
            second = second.next;
        }
        second.next = second.next.next;
        return dummy.next;
    }
}
```

6. Reorder List

PROBLEM DEFINITION

You are given the head of a singly linked list. Reorder the list to alternate between the start and end nodes.

PRIMARY TEST CASES

Input: head = [1,2,3,4]
Output: [1,4,2,3]

APPROACH & KEY INSIGHTS

1. Find middle point. 2. Reverse the entire second half segment. 3. Merge alternating nodes sequentially.

PERFECT JAVA SOLUTION

```
class Solution {
    public void reorderList(ListNode head) {
        if (head == null || head.next == null) return;
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        ListNode prev = null, curr = slow.next;
        slow.next = null;
        while (curr != null) {
            ListNode next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }
        ListNode first = head, second = prev;
        while (second != null) {
            ListNode t1 = first.next, t2 = second.next;
            first.next = second;
            second.next = t1;
            first = t1;
            second = t2;
        }
    }
}
```

7. LRU Cache

PROBLEM DEFINITION

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.

PRIMARY TEST CASES

```
LRUCache cache = new LRUCache(2); cache.put(1, 1); cache.get(1);
```

APPROACH & KEY INSIGHTS

Combine a HashMap providing $O(1)$ lookups with a custom Doubly Linked List tracking access history structures cleanly.

PERFECT JAVA SOLUTION

```
class LRUCache {
    class Node { int key, val; Node prev, next; }
    private Map map = new HashMap<>();
    private Node head, tail;
    private int capacity;
    public LRUCache(int capacity) {
        this.capacity = capacity;
        head = new Node(); tail = new Node();
        head.next = tail; tail.prev = head;
    }
    public int get(int key) {
        Node node = map.get(key);
        if (node == null) return -1;
        moveToHead(node);
        return node.val;
    }
    public void put(int key, int value) {
        Node node = map.get(key);
        if (node != null) {
            node.val = value;
            moveToHead(node);
        } else {
            Node newNode = new Node();
            newNode.key = key; newNode.val = value;
            map.put(key, newNode);
            addNode(newNode);
            if (map.size() > capacity) {
                Node tailNode = popTail();
                map.remove(tailNode.key);
            }
        }
    }
    private void addNode(Node node) {
        node.prev = head; node.next = head.next;
        head.next.prev = node; head.next = node;
    }
    private void removeNode(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }
}
```

```
}  
private void moveToHead(Node node) { removeNode(node); addNode(node); }  
private Node popTail() { Node res = tail.prev; removeNode(res); return res; }  
}
```

4. Stack & Queue

1. Valid Parentheses

PROBLEM DEFINITION

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

PRIMARY TEST CASES

```
Input: s = "()[]{}"  
Output: true
```

APPROACH & KEY INSIGHTS

Push expected closing characters when encountering opening boundaries, checking items match stack tops sequentially.

PERFECT JAVA SOLUTION

```
class Solution {  
    public boolean isValid(String s) {  
        Stack stack = new Stack<>();  
        for (char c : s.toCharArray()) {  
            if (c == '(') stack.push(')');  
            else if (c == '{') stack.push('}');  
            else if (c == '[') stack.push(']');  
            else if (stack.isEmpty() || stack.pop() != c) return false;  
        }  
        return stack.isEmpty();  
    }  
}
```


2. Implement Queue using Stacks

PROBLEM DEFINITION

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (`push`, `peek`, `pop`, and `empty`).

PRIMARY TEST CASES

```
MyQueue q = new MyQueue(); q.push(1); q.pop();
```

APPROACH & KEY INSIGHTS

Maintain two distinct internal collections: `input` and `output`. Flush `input` structures completely into `output` only when `output` runs dry.

PERFECT JAVA SOLUTION

```
class MyQueue {
    private Stack in = new Stack<>();
    private Stack out = new Stack<>();
    public void push(int x) { in.push(x); }
    public int pop() { peek(); return out.pop(); }
    public int peek() {
        if (out.isEmpty()) {
            while (!in.isEmpty()) out.push(in.pop());
        }
        return out.peek();
    }
    public boolean empty() { return in.isEmpty() && out.isEmpty(); }
}
```

3. Min Stack

PROBLEM DEFINITION

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

PRIMARY TEST CASES

```
MinStack ms = new MinStack(); ms.push(-2); ms.getMin();
```

APPROACH & KEY INSIGHTS

Keep a secondary underlying track system syncing baseline bounds or encapsulate values directly inside unified tracking frames.

PERFECT JAVA SOLUTION

```
class MinStack {
    private Stack st = new Stack<>();
    private Stack minSt = new Stack<>();
    public void push(int val) {
        st.push(val);
        if (minSt.isEmpty() || val <= minSt.peek()) minSt.push(val);
    }
    public void pop() {
        if (st.pop().equals(minSt.peek())) minSt.pop();
    }
    public int top() { return st.peek(); }
    public int getMin() { return minSt.peek(); }
}
```

4. Daily Temperatures

PROBLEM DEFINITION

Given an array of integers `temperatures` represents the daily temperatures, return *an array* `answer` *such that* `answer[i]` *is the number of days you have to wait after the* `i-th` *day to get a warmer temperature*.

PRIMARY TEST CASES

```
Input: temperatures = [73,74,75,71,69,72,76,73]
Output: [1,1,4,2,1,1,0,0]
```

APPROACH & KEY INSIGHTS

Maintain a Monotonic Decreasing Stack storing historical elements' array index boundaries until hot bounds clear lower states.

PERFECT JAVA SOLUTION

```
class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        int[] ans = new int[temperatures.length];
        Stack st = new Stack<>();
        for (int i = 0; i < temperatures.length; i++) {
            while (!st.isEmpty() && temperatures[i] > temperatures[st.peek()]) {
                int idx = st.pop();
                ans[idx] = i - idx;
            }
            st.push(i);
        }
        return ans;
    }
}
```

5. Next Greater Element I

PROBLEM DEFINITION

The **next greater element** of some element `x` in an array is the first greater element that is to the right of `x` in the same array.

PRIMARY TEST CASES

Input: nums1 = [4,1,2], nums2 = [1,3,4,2]
Output: [-1,3,-1]

APPROACH & KEY INSIGHTS

Precompute all target limits across the broader source array using a monotonic stack combined with mapped record key captures.

PERFECT JAVA SOLUTION

```
class Solution {
    public int[] nextGreaterElement(int[] nums1, int[] nums2) {
        Map map = new HashMap<>();
        Stack stack = new Stack<>();
        for (int num : nums2) {
            while (!stack.isEmpty() && stack.peek() < num) map.put(stack.pop(), num);
            stack.push(num);
        }
        int[] ans = new int[nums1.length];
        for (int i = 0; i < nums1.length; i++) ans[i] = map.getOrDefault(nums1[i], -1);
        return ans;
    }
}
```

6. Evaluate Reverse Polish Notation

PROBLEM DEFINITION

Evaluate the value of an arithmetic expression in Reverse Polish Notation (Postfix Expression). Valid operators are `+`, `-`, `*`, and `/`.

PRIMARY TEST CASES

Input: tokens = ["2", "1", "+", "3", "*"]
Output: 9

APPROACH & KEY INSIGHTS

Process items sequentially: push plain numerals directly onto the stack, and pop the top two values to apply operations when an operator is encountered.

PERFECT JAVA SOLUTION

```
class Solution {
    public int evalRPN(String[] tokens) {
        Stack st = new Stack<>();
        for (String t : tokens) {
            if ("+-*/".contains(t)) {
                int b = st.pop(), a = st.pop();
                if (t.equals("+")) st.push(a + b);
                else if (t.equals("-")) st.push(a - b);
                else if (t.equals("*")) st.push(a * b);
                else st.push(a / b);
            } else st.push(Integer.parseInt(t));
        }
        return st.pop();
    }
}
```

7. Sliding Window Maximum

PROBLEM DEFINITION

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right.

PRIMARY TEST CASES

Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]

APPROACH & KEY INSIGHTS

Use a Monotonic Deque containing elements indices, keeping them sorted dynamically to ensure the maximum item index always sits out front.

PERFECT JAVA SOLUTION

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int n = nums.length;
        int[] ans = new int[n - k + 1];
        Deque dq = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            if (!dq.isEmpty() && dq.peekFirst() < i - k + 1) dq.pollFirst();
            while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) dq.pollLast();
            dq.offerLast(i);
            if (i >= k - 1) ans[i - k + 1] = nums[dq.peekFirst()];
        }
        return ans;
    }
}
```

5. Trees

1. Maximum Depth of Binary Tree

PROBLEM DEFINITION

Given the `root` of a binary tree, return *its maximum depth*.

PRIMARY TEST CASES

Input: root = [3,9,20,null,null,15,7]
Output: 3

APPROACH & KEY INSIGHTS

Evaluate standard post-order tree path traversals: `1 + Math.max(maxDepth(left), maxDepth(right))` recursively.

PERFECT JAVA SOLUTION

```
class Solution {
    public int maxDepth(TreeNode root) {
        if (root == null) return 0;
        return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
    }
}
```

| 2. Same Tree

PROBLEM DEFINITION

Given the roots of two binary trees `p` and `q`, write a function to check if they are the same or not.

PRIMARY TEST CASES

Input: p = [1,2,3], q = [1,2,3]
Output: true

APPROACH & KEY INSIGHTS

Recursively check matching nodes: both nodes must be null, or both must be non-null with equal values and identical structural layouts.

PERFECT JAVA SOLUTION

```
class Solution {  
    public boolean isSameTree(TreeNode p, TreeNode q) {  
        if (p == null && q == null) return true;  
        if (p == null || q == null) return false;  
        return (p.val == q.val) && isSameTree(p.left, q.left) && isSameTree(p.right,  
q.right);  
    }  
}
```


3. Invert Binary Tree

PROBLEM DEFINITION

Given the `root` of a binary tree, invert the tree, and return *its root*.

PRIMARY TEST CASES

Input: root = [4,2,7,1,3,6,9]
Output: [4,7,2,9,6,3,1]

APPROACH & KEY INSIGHTS

Invert subtrees recursively by swapping the left and right child nodes at the current level, then continuing down.

PERFECT JAVA SOLUTION

```
class Solution {  
    public TreeNode invertTree(TreeNode root) {  
        if (root == null) return null;  
        TreeNode temp = root.left;  
        root.left = invertTree(root.right);  
        root.right = invertTree(temp);  
        return root;  
    }  
}
```

4. Binary Tree Level Order Traversal

PROBLEM DEFINITION

Given the `root` of a binary tree, return *the level order traversal of its nodes' values*. (i.e., from left to right, level by level).

PRIMARY TEST CASES

```
Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]
```

APPROACH & KEY INSIGHTS

Employ Breadth-First Search using a Queue structure. Read all elements current level sizes sequentially to cluster output loops.

PERFECT JAVA SOLUTION

```
class Solution {
    public List<Integer> levelOrder(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        if (root == null) return ans;
        Queue<TreeNode> q = new LinkedList<>();
        q.add(root);
        while (!q.isEmpty()) {
            int size = q.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode curr = q.poll();
                level.add(curr.val);
                if (curr.left != null) q.add(curr.left);
                if (curr.right != null) q.add(curr.right);
            }
            ans.add(level);
        }
        return ans;
    }
}
```

5. Lowest Common Ancestor of a Binary Tree

PROBLEM DEFINITION

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

PRIMARY TEST CASES

Input: root = [3,5,1...], p = 5, q = 1
Output: 3

APPROACH & KEY INSIGHTS

Search both left and right branches recursively. If a branch returns a match for either node, pass that node upward to find the intersection.

PERFECT JAVA SOLUTION

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null || root == p || root == q) return root;
        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);
        if (left != null && right != null) return root;
        return left != null ? left : right;
    }
}
```

6. Validate Binary Search Tree

PROBLEM DEFINITION

Given the `root` of a binary tree, determine if it is a valid binary search tree (BST).

PRIMARY TEST CASES

Input: root = [2,1,3]
Output: true

APPROACH & KEY INSIGHTS

Validate values recursively using strict upper and lower boundary thresholds passed down across child nodes.

PERFECT JAVA SOLUTION

```
class Solution {
    public boolean isValidBST(TreeNode root) {
        return validate(root, null, null);
    }
    private boolean validate(TreeNode node, Integer low, Integer high) {
        if (node == null) return true;
        if ((low != null && node.val <= low) || (high != null && node.val >= high)) return
false;
        return validate(node.left, low, node.val) && validate(node.right, node.val, high);
    }
}
```

7. Serialize and Deserialize Binary Tree

PROBLEM DEFINITION

Design an algorithm to serialize and deserialize a binary tree.

PRIMARY TEST CASES

Serialization format: "1,2,##,3,4,##,5,##"

APPROACH & KEY INSIGHTS

Use pre-order traversal routines. Output specific unique non-numeric placeholder tags like `#` to represent null fields accurately.

PERFECT JAVA SOLUTION

```
public class Codec {
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        buildString(root, sb);
        return sb.toString();
    }
    private void buildString(TreeNode node, StringBuilder sb) {
        if (node == null) sb.append("#,");
        else {
            sb.append(node.val).append(",");
            buildString(node.left, sb);
            buildString(node.right, sb);
        }
    }
    public TreeNode deserialize(String data) {
        Queue nodes = new LinkedList<>(Arrays.asList(data.split(",")));
        return buildTree(nodes);
    }
    private TreeNode buildTree(Queue nodes) {
        String val = nodes.poll();
        if (val.equals("#")) return null;
        TreeNode node = new TreeNode(Integer.parseInt(val));
        node.left = buildTree(nodes);
        node.right = buildTree(nodes);
        return node;
    }
}
```

6. Binary Search

| 1. Binary Search

PROBLEM DEFINITION

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`.

PRIMARY TEST CASES

Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4

APPROACH & KEY INSIGHTS

Maintain `left` and `right` pointers. Calculate the middle index `mid` to continually divide the sorted search space in half.

PERFECT JAVA SOLUTION

```
class Solution {
    public int search(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) return mid;
            else if (nums[mid] < target) left = mid + 1;
            else right = mid - 1;
        }
        return -1;
    }
}
```

2. Search Insert Position

PROBLEM DEFINITION

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

PRIMARY TEST CASES

Input: nums = [1,3,5,6], target = 5
Output: 2

APPROACH & KEY INSIGHTS

Perform a standard binary search. If the target is not found, the `left` pointer will naturally settle at the correct insertion index.

PERFECT JAVA SOLUTION

```
class Solution {  
    public int searchInsert(int[] nums, int target) {  
        int left = 0, right = nums.length - 1;  
        while (left <= right) {  
            int mid = left + (right - left) / 2;  
            if (nums[mid] == target) return mid;  
            else if (nums[mid] < target) left = mid + 1;  
            else right = mid - 1;  
        }  
        return left;  
    }  
}
```

3. First Bad Version

PROBLEM DEFINITION

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

PRIMARY TEST CASES

Input: n = 5, bad = 4
Output: 4

APPROACH & KEY INSIGHTS

Binary search across versions. If `isBadVersion(mid)` is true, look left by setting `right = mid`. Otherwise, look right with `left = mid + 1`.

PERFECT JAVA SOLUTION

```
public class Solution extends VersionControl {
    public int firstBadVersion(int n) {
        int left = 1, right = n;
        while (left < right) {
            int mid = left + (right - left) / 2;
            if (isBadVersion(mid)) right = mid;
            else left = mid + 1;
        }
        return left;
    }
}
```


4. Search in Rotated Sorted Array

PROBLEM DEFINITION

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of* `target` *if it is in* `nums`, or* `-1` *if it is not in* `nums`.

PRIMARY TEST CASES

Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4

APPROACH & KEY INSIGHTS

Identify which half of the array is normally sorted at the `mid` position, then check if the target falls within that sorted range.

PERFECT JAVA SOLUTION

```
class Solution {
    public int search(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) return mid;
            if (nums[left] <= nums[mid]) {
                if (target >= nums[left] && target < nums[mid]) right = mid - 1;
                else left = mid + 1;
            } else {
                if (target > nums[mid] && target <= nums[right]) left = mid + 1;
                else right = mid - 1;
            }
        }
        return -1;
    }
}
```

5. Find Peak Element

PROBLEM DEFINITION

A peak element is an element that is strictly greater than its neighbors. Given a **0-indexed** integer array `nums`, find a peak element, and return its index.

PRIMARY TEST CASES

Input: `nums = [1,2,3,1]`
Output: 2

APPROACH & KEY INSIGHTS

If `nums[mid] < nums[mid + 1]`, a peak must lie to the right, so move `left = mid + 1`. Otherwise, the peak lies to the left or at `mid`.

PERFECT JAVA SOLUTION

```
class Solution {
    public int findPeakElement(int[] nums) {
        int left = 0, right = nums.length - 1;
        while (left < right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] < nums[mid + 1]) left = mid + 1;
            else right = mid;
        }
        return left;
    }
}
```

6. Koko Eating Bananas

PROBLEM DEFINITION

Koko loves to eat bananas. There are n piles of bananas, the i -th pile has $piles[i]$ bananas. The guards have gone and will come back in h hours.

PRIMARY TEST CASES

Input: $piles = [3, 6, 7, 11]$, $h = 8$
Output: 4

APPROACH & KEY INSIGHTS

Binary search over the possible eating speeds, ranging from 1 to the maximum pile size. Check the total hours required for each candidate speed.

PERFECT JAVA SOLUTION

```
class Solution {
    public int minEatingSpeed(int[] piles, int h) {
        int left = 1, right = 0;
        for (int p : piles) right = Math.max(right, p);
        while (left < right) {
            int mid = left + (right - left) / 2;
            int hours = 0;
            for (int p : piles) hours += (p + mid - 1) / mid;
            if (hours <= h) right = mid;
            else left = mid + 1;
        }
        return left;
    }
}
```

7. Median of Two Sorted Arrays

PROBLEM DEFINITION

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return **the median** of the two sorted arrays. The overall run time complexity should be $O(\log(m+n))$.

PRIMARY TEST CASES

Input: nums1 = [1,3], nums2 = [2]
Output: 2.00000

APPROACH & KEY INSIGHTS

Binary search on the partition point of the smaller array. Ensure elements on the left side are smaller than or equal to elements on the right.

PERFECT JAVA SOLUTION

```
class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        if (nums1.length > nums2.length) return findMedianSortedArrays(nums2, nums1);
        int x = nums1.length, y = nums2.length, left = 0, right = x;
        while (left <= right) {
            int px = left + (right - left) / 2;
            int py = (x + y + 1) / 2 - px;
            int maxLx = (px == 0) ? Integer.MIN_VALUE : nums1[px - 1];
            int minRx = (px == x) ? Integer.MAX_VALUE : nums1[px];
            int maxLy = (py == 0) ? Integer.MIN_VALUE : nums2[py - 1];
            int minRy = (py == y) ? Integer.MAX_VALUE : nums2[py];
            if (maxLx <= minRy && maxLy <= minRx) {
                if ((x + y) % 2 == 0) return (Math.max(maxLx, maxLy) + Math.min(minRx,
minRy)) / 2.0;
                else return Math.max(maxLx, maxLy);
            } else if (maxLx > minRy) right = px - 1;
            else left = px + 1;
        }
        return 0.0;
    }
}
```

7. Heap / Priority Queue

| 1. Kth Largest Element in an Array

PROBLEM DEFINITION

Given an integer array `nums` and an integer `k`, return *the* `k-th` *largest element in the array*.

PRIMARY TEST CASES

Input: nums = [3,2,1,5,6,4], k = 2
Output: 5

APPROACH & KEY INSIGHTS

Maintain a Min-Heap of size `k`. Elements are added one by one; when the size exceeds `k`, the smallest element is evicted, leaving the k-th largest at the top.

PERFECT JAVA SOLUTION

```
class Solution {  
    public int findKthLargest(int[] nums, int k) {  
        PriorityQueue pq = new PriorityQueue<>();  
        for (int num : nums) {  
            pq.add(num);  
            if (pq.size() > k) pq.poll();  
        }  
        return pq.peek();  
    }  
}
```

2. Top K Frequent Elements

PROBLEM DEFINITION

Given an integer array `nums` and an integer `k`, return *the* `k` *most frequent elements*. You may return the answer in **any order**.

PRIMARY TEST CASES

Input: nums = [1,1,1,2,2,3], k = 2
Output: [1,2]

APPROACH & KEY INSIGHTS

Count frequencies using a HashMap, then add the unique elements into a Min-Heap ordered by frequency, keeping the heap size capped at `k`.

PERFECT JAVA SOLUTION

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map map = new HashMap<>();
        for (int n : nums) map.put(n, map.getOrDefault(n, 0) + 1);
        PriorityQueue pq = new PriorityQueue<>((a, b) -> map.get(a) - map.get(b));
        for (int n : map.keySet()) {
            pq.add(n);
            if (pq.size() > k) pq.poll();
        }
        int[] ans = new int[k];
        for (int i = 0; i < k; i++) ans[i] = pq.poll();
        return ans;
    }
}
```

3. Merge K Sorted Lists

PROBLEM DEFINITION

You are given an array of `k` linked-lists `lists`, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

PRIMARY TEST CASES

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
```

APPROACH & KEY INSIGHTS

Add the head nodes of all `k` lists into a Min-Heap. Repeatedly extract the minimum node, append it to the result list, and insert its next node into the heap.

PERFECT JAVA SOLUTION

```
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        PriorityQueue pq = new PriorityQueue<>((a, b) -> a.val - b.val);
        for (ListNode l : lists) if (l != null) pq.add(l);
        ListNode dummy = new ListNode(0), curr = dummy;
        while (!pq.isEmpty()) {
            ListNode node = pq.poll();
            curr.next = node;
            curr = curr.next;
            if (node.next != null) pq.add(node.next);
        }
        return dummy.next;
    }
}
```

4. Find Median from Data Stream

PROBLEM DEFINITION

The **median** is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

PRIMARY TEST CASES

```
MedianFinder mf = new MedianFinder(); mf.addNum(1); mf.addNum(2); mf.findMedian();
```

APPROACH & KEY INSIGHTS

Use two heaps: a Max-Heap for the smaller half of the numbers and a Min-Heap for the larger half. Balance their sizes so they differ by at most one.

PERFECT JAVA SOLUTION

```
class MedianFinder {
    private PriorityQueue small = new PriorityQueue<>(Collections.reverseOrder());
    private PriorityQueue large = new PriorityQueue<>();
    public void addNum(int num) {
        small.add(num);
        large.add(small.poll());
        if (small.size() < large.size()) small.add(large.poll());
    }
    public double findMedian() {
        return small.size() > large.size() ? small.peek() : (small.peek() +
        large.peek()) / 2.0;
    }
}
```


5. Task Scheduler

PROBLEM DEFINITION

Given a characters array `tasks`, representing the tasks a CPU needs to do, where each character represents a different task. Tasks could be done in any order. Each task is done in one unit of time. For each unit of time, the CPU could complete either one task or just be idle.

PRIMARY TEST CASES

Input: tasks = ["A","A","A","B","B","B"], n = 2
Output: 8

APPROACH & KEY INSIGHTS

Calculate the frequency of the most frequent task. The maximum possible idle slots needed depends on this count and the cooling period `n`.

PERFECT JAVA SOLUTION

```
class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] freq = new int[26];
        for (char c : tasks) freq[c - 'a']++;
        Arrays.sort(freq);
        int maxFreq = freq[25];
        int idleTime = (maxFreq - 1) * n;
        for (int i = 24; i >= 0 && freq[i] > 0; i--) {
            idleTime -= Math.min(maxFreq - 1, freq[i]);
        }
        return tasks.length + Math.max(0, idleTime);
    }
}
```

6. Reorganize String

PROBLEM DEFINITION

Given a string `s`, rearrange the characters of `s` so that any two adjacent characters are not the same.

PRIMARY TEST CASES

Input: s = "aab"
Output: "aba"

APPROACH & KEY INSIGHTS

Store character frequencies in a Max-Heap. At each step, extract the two most frequent distinct characters, append them to the result, and reinsert them with decremented counts.

PERFECT JAVA SOLUTION

```
class Solution {
    public String reorganizeString(String s) {
        int[] counts = new int[26];
        for (char c : s.toCharArray()) counts[c - 'a']++;
        PriorityQueue pq = new PriorityQueue<>((a, b) -> b[1] - a[1]);
        for (int i = 0; i < 26; i++) if (counts[i] > 0) pq.add(new int[]{i + 'a', counts[i]});
        StringBuilder sb = new StringBuilder();
        while (pq.size() >= 2) {
            int[] first = pq.poll(), second = pq.poll();
            sb.append((char) first[0]).append((char) second[0]);
            if (--first[1] > 0) pq.add(first);
            if (--second[1] > 0) pq.add(second);
        }
        if (!pq.isEmpty()) {
            int[] last = pq.poll();
            if (last[1] > 1) return "";
            sb.append((char) last[0]);
        }
        return sb.toString();
    }
}
```

7. IPO

PROBLEM DEFINITION

Suppose LeetCode will start its IPO soon. In order to sell a good price of its shares to Venture Capital, LeetCode would like to work on some projects to increase its capital before the IPO.

PRIMARY TEST CASES

Input: k = 2, w = 0, profits = [1,2,3], capital = [0,1,1]
Output: 4

APPROACH & KEY INSIGHTS

Sort projects by required capital. Use a Max-Heap to track the profits of all affordable projects given the current capital, selecting the most profitable project at each step.

PERFECT JAVA SOLUTION

```
class Solution {
    public int findMaximizedCapital(int k, int w, int[] profits, int[] capital) {
        int n = profits.length;
        int[][] projects = new int[n][2];
        for (int i = 0; i < n; i++) projects[i] = new int[]{capital[i], profits[i]};
        Arrays.sort(projects, (a, b) -> a[0] - b[0]);
        PriorityQueue pq = new PriorityQueue<>(Collections.reverseOrder());
        int idx = 0;
        for (int i = 0; i < k; i++) {
            while (idx < n && projects[idx][0] <= w) pq.add(projects[idx++][1]);
            if (pq.isEmpty()) break;
            w += pq.poll();
        }
        return w;
    }
}
```

8. Graphs

1. Number of Islands

PROBLEM DEFINITION

Given an $m \times n$ 2D binary grid `grid` which represents a map of `'1'`s (land) and `'0'`s (water), return *the number of islands*.

PRIMARY TEST CASES

```
Input: grid = [{"1","1","0"}, {"0","0","1"}]
Output: 2
```

APPROACH & KEY INSIGHTS

Iterate through the grid. When land (`'1'`) is found, increment the counter and launch a DFS/BFS to flip all connected land tiles to water (`'0'`) to avoid re-counting.

PERFECT JAVA SOLUTION

```
class Solution {
    public int numIslands(char[][] grid) {
        int count = 0;
        for (int i = 0; i < grid.length; i++) {
            for (int j = 0; j < grid[0].length; j++) {
                if (grid[i][j] == '1') {
                    dfs(grid, i, j);
                    count++;
                }
            }
        }
        return count;
    }
    private void dfs(char[][] grid, int r, int c) {
        if (r < 0 || c < 0 || r >= grid.length || c >= grid[0].length || grid[r][c] == '0') return;
        grid[r][c] = '0';
        dfs(grid, r + 1, c); dfs(grid, r - 1, c);
        dfs(grid, r, c + 1); dfs(grid, r, c - 1);
    }
}
```

| 2. Clone Graph

PROBLEM DEFINITION

Given a reference of a node in a **connected** undirected graph. Return a **deep copy** (clone) of the graph.

PRIMARY TEST CASES

```
Input: adjList = [[2,4],[1,3],[2,4],[1,3]]
Output: Cloned graph matching layout structure.
```

APPROACH & KEY INSIGHTS

Traverse the graph using DFS or BFS while mapping each original node to its newly cloned counterpart in a HashMap to handle cyclic references cleanly.

PERFECT JAVA SOLUTION

```
class Solution {
    private Map map = new HashMap<>();
    public Node cloneGraph(Node node) {
        if (node == null) return null;
        if (map.containsKey(node)) return map.get(node);
        Node clone = new Node(node.val);
        map.put(node, clone);
        for (Node neighbor : node.neighbors) {
            clone.neighbors.add(cloneGraph(neighbor));
        }
        return clone;
    }
}
```

3. Flood Fill

PROBLEM DEFINITION

An image is represented by an $m \times n$ integer grid `image` where `image[i][j]` represents the pixel value of the image. You are also given three integers `sr`, `sc`, and `color`.

PRIMARY TEST CASES

```
Input: image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, color = 2
Output: [[2,2,2],[2,2,0],[2,0,1]]
```

APPROACH & KEY INSIGHTS

Launch a basic recursive DFS starting from the pixel `(sr, sc)`. If adjacent target elements match the initial starting color, update them to the new color.

PERFECT JAVA SOLUTION

```
class Solution {
    public int[][] floodFill(int[][] image, int sr, int sc, int color) {
        int oldColor = image[sr][sc];
        if (oldColor != color) dfs(image, sr, sc, oldColor, color);
        return image;
    }
    private void dfs(int[][] img, int r, int c, int oldColor, int newColor) {
        if (r < 0 || c < 0 || r >= img.length || c >= img[0].length || img[r][c] !=
oldColor) return;
        img[r][c] = newColor;
        dfs(img, r + 1, c, oldColor, newColor);
        dfs(img, r - 1, c, oldColor, newColor);
        dfs(img, r, c + 1, oldColor, newColor);
        dfs(img, r, c - 1, oldColor, newColor);
    }
}
```

4. Course Schedule

PROBLEM DEFINITION

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [a\_i, b\_i]` indicates that you must take course `b\_i` first if you want to take course `a\_i`.

PRIMARY TEST CASES

```
Input: numCourses = 2, prerequisites = [[1,0]]
Output: true
```

APPROACH & KEY INSIGHTS

Detect cycles within a directed graph using Kahn's algorithm (indegree-based BFS) or typical DFS color-marking schemes. If a cycle is present, topological sorting is impossible.

PERFECT JAVA SOLUTION

```
class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        int[] indegree = new int[numCourses];
        List<Integer> adj = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
        for (int[] pre : prerequisites) {
            adj.get(pre[1]).add(pre[0]);
            indegree[pre[0]]++;
        }
        Queue<Integer> q = new LinkedList<>();
        for (int i = 0; i < numCourses; i++) if (indegree[i] == 0) q.add(i);
        int count = 0;
        while (!q.isEmpty()) {
            int curr = q.poll();
            count++;
            for (int next : adj.get(curr)) {
                if (--indegree[next] == 0) q.add(next);
            }
        }
        return count == numCourses;
    }
}
```

5. Pacific Atlantic Water Flow

PROBLEM DEFINITION

There is an $m \times n$ rectangular island that borders both the **Pacific Ocean** and **Atlantic Ocean**. The Pacific Ocean touches the island's left and top edges, and the Atlantic Ocean touches the island's right and bottom edges.

PRIMARY TEST CASES

Input: heights = [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1]]
Output: Coordinates that flow to both oceans.

APPROACH & KEY INSIGHTS

Run two separate DFS pointer passes starting from the ocean boundaries moving inward up ascending heights. The cells visited by both passes form the result set.

PERFECT JAVA SOLUTION

```
class Solution {
    public List<int[]> pacificAtlantic(int[][] heights) {
        List<int[]> res = new ArrayList<>();
        int m = heights.length, n = heights[0].length;
        boolean[][] pac = new boolean[m][n], atl = new boolean[m][n];
        for (int i = 0; i < m; i++) { dfs(heights, i, 0, pac, 0); dfs(heights, i, n - 1, atl, 0); }
        for (int j = 0; j < n; j++) { dfs(heights, 0, j, pac, 0); dfs(heights, m - 1, j, atl, 0); }
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (pac[i][j] && atl[i][j]) res.add(Arrays.asList(i, j));
            }
        }
        return res;
    }
    private void dfs(int[][] h, int r, int c, boolean[][] visit, int prev) {
        if (r < 0 || c < 0 || r >= h.length || c >= h[0].length || visit[r][c] || h[r][c] < prev) return;
        visit[r][c] = true;
        dfs(h, r + 1, c, visit, h[r][c]); dfs(h, r - 1, c, visit, h[r][c]);
        dfs(h, r, c + 1, visit, h[r][c]); dfs(h, r, c - 1, visit, h[r][c]);
    }
}
```


6. Word Ladder

PROBLEM DEFINITION

A **transformation sequence** from a word `beginWord` to a word `endWord` using a dictionary `wordList` is a sequence of words such that each adjacent pair of words differs by a single character.

PRIMARY TEST CASES

```
Input: beginWord = "hit", endWord = "cog", wordList =
["hot", "dot", "dog", "lot", "log", "cog"]
Output: 5
```

APPROACH & KEY INSIGHTS

Find the shortest path using Breadth-First Search (BFS). At each step, mutate every character position from 'a' to 'z' to discover valid transitions within the word set.

PERFECT JAVA SOLUTION

```
class Solution {
    public int ladderLength(String beginWord, String endWord, List wordList) {
        Set set = new HashSet<>(wordList);
        if (!set.contains(endWord)) return 0;
        Queue q = new LinkedList<>();
        q.add(beginWord);
        int level = 1;
        while (!q.isEmpty()) {
            int size = q.size();
            for (int i = 0; i < size; i++) {
                String curr = q.poll();
                if (curr.equals(endWord)) return level;
                char[] chars = curr.toCharArray();
                for (int j = 0; j < chars.length; j++) {
                    char orig = chars[j];
                    for (char c = 'a'; c <= 'z'; c++) {
                        chars[j] = c;
                        String next = new String(chars);
                        if (set.contains(next)) {
                            q.add(next);
                            set.remove(next);
                        }
                    }
                    chars[j] = orig;
                }
            }
            level++;
        }
        return 0;
    }
}
```

7. Network Delay Time

PROBLEM DEFINITION

You are given a network of n nodes, labeled from 1 to n . You are also given times , a list of travel times as directed edges $\text{times}[i] = (u_i, v_i, w_i)$.

PRIMARY TEST CASES

Input: $\text{times} = [[2,1,1],[2,3,1],[3,4,1]]$, $n = 4$, $k = 2$
Output: 2

APPROACH & KEY INSIGHTS

Apply Dijkstra's algorithm using a Min-Priority Queue to compute the shortest path weights from source node k to all other reachable nodes.

PERFECT JAVA SOLUTION

```
class Solution {
    public int networkDelayTime(int[][] times, int n, int k) {
        Map<Integer, List<Integer>> graph = new HashMap<>();
        for (int[] t : times) {
            graph.computeIfAbsent(t[0], x -> new ArrayList<>()).add(new int[]{t[1],
t[2]});
        }
        PriorityQueue<Integer> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        pq.add(new int[]{k, 0});
        Map<Integer, Integer> dist = new HashMap<>();
        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int node = curr[0], d = curr[1];
            if (dist.containsKey(node)) continue;
            dist.put(node, d);
            if (graph.containsKey(node)) {
                for (int[] edge : graph.get(node)) {
                    if (!dist.containsKey(edge[0])) pq.add(new int[]{edge[0], d +
edge[1]});
                }
            }
        }
        if (dist.size() != n) return -1;
        int max = 0;
        for (int d : dist.values()) max = Math.max(max, d);
        return max;
    }
}
```

9. Dynamic Programming

| 1. Climbing Stairs

PROBLEM DEFINITION

You are climbing a staircase. It takes `n` steps to reach the top. Each time you can either climb `1` or `2` steps. In how many distinct ways can you climb to the top?

PRIMARY TEST CASES

```
Input: n = 3
Output: 3
```

APPROACH & KEY INSIGHTS

The number of ways to reach step `n` is the sum of the ways to reach `n-1` and `n-2`. Optimize space by tracking just the last two calculated step counts.

PERFECT JAVA SOLUTION

```
class Solution {
    public int climbStairs(int n) {
        if (n <= 2) return n;
        int first = 1, second = 2;
        for (int i = 3; i <= n; i++) {
            int third = first + second;
            first = second;
            second = third;
        }
        return second;
    }
}
```

2. House Robber

PROBLEM DEFINITION

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected.

PRIMARY TEST CASES

Input: nums = [1,2,3,1]
Output: 4

APPROACH & KEY INSIGHTS

For each house, choose the maximum value between robbing it (current value + maximum from two houses back) or skipping it (maximum value up to the previous house).

PERFECT JAVA SOLUTION

```
class Solution {
    public int rob(int[] nums) {
        int prev1 = 0, prev2 = 0;
        for (int num : nums) {
            int temp = prev1;
            prev1 = Math.max(prev2 + num, prev1);
            prev2 = temp;
        }
        return prev1;
    }
}
```

3. Coin Change

PROBLEM DEFINITION

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

PRIMARY TEST CASES

```
Input: coins = [1,2,5], amount = 11
Output: 3
```

APPROACH & KEY INSIGHTS

Build a 1D DP table up to `amount`. For each value, minimize the coins needed by checking `dp[i - coin] + 1` for all available coin options.

PERFECT JAVA SOLUTION

```
class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;
        for (int i = 1; i <= amount; i++) {
            for (int coin : coins) {
                if (i >= coin) dp[i] = Math.min(dp[i], dp[i - coin] + 1);
            }
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
}
```

4. Longest Increasing Subsequence

PROBLEM DEFINITION

Given an integer array `nums`, return *the length of the longest strictly increasing subsequence*.

PRIMARY TEST CASES

Input: nums = [10,9,2,5,3,7,101,18]
Output: 4

APPROACH & KEY INSIGHTS

Maintain a dynamic list tracking the smallest tail end values for all increasing subsequences found. Update elements using binary search via `Arrays.binarySearch`.

PERFECT JAVA SOLUTION

```
class Solution {
    public int lengthOfLIS(int[] nums) {
        int[] tails = new int[nums.length];
        int size = 0;
        for (int x : nums) {
            int i = 0, j = size;
            while (i != j) {
                int mid = (i + j) / 2;
                if (tails[mid] < x) i = mid + 1;
                else j = mid;
            }
            tails[i] = x;
            if (i == size) size++;
        }
        return size;
    }
}
```

5. Partition Equal Subset Sum

PROBLEM DEFINITION

Given an integer array `nums`, return `true` *if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or* `false` *otherwise*.

PRIMARY TEST CASES

Input: nums = [1,5,11,5]
Output: true

APPROACH & KEY INSIGHTS

Ensure the total array sum is even, then solve the 0/1 knapsack subproblem to determine if a subset can sum exactly to $\text{totalSum} / 2$.

PERFECT JAVA SOLUTION

```
class Solution {
    public boolean canPartition(int[] nums) {
        int sum = 0;
        for (int num : nums) sum += num;
        if (sum % 2 != 0) return false;
        int target = sum / 2;
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;
        for (int num : nums) {
            for (int j = target; j >= num; j--) {
                if (dp[j - num]) dp[j] = true;
            }
        }
        return dp[target];
    }
}
```

6. Longest Common Subsequence

PROBLEM DEFINITION

Given two strings `text1` and `text2`, return *the length of their longest common subsequence*. If there is no common subsequence, return `0`.

PRIMARY TEST CASES

Input: text1 = "abcde", text2 = "ace"
Output: 3

APPROACH & KEY INSIGHTS

Use a 2D DP matrix to compare characters. If they match, increment from the diagonal element: $dp[i][j] = dp[i-1][j-1] + 1$. Otherwise, take the maximum from the top or left cells.

PERFECT JAVA SOLUTION

```
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length(), n = text2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1.charAt(i - 1) == text2.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1] + 1;
                else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
        return dp[m][n];
    }
}
```


7. Edit Distance

PROBLEM DEFINITION

Given two strings `word1` and `word2`, return *the minimum number of operations required to convert `word1` to `word2`*. You have the following three operations permitted on a word: Insert, Delete, Replace.

PRIMARY TEST CASES

```
Input: word1 = "horse", word2 = "ros"
Output: 3
```

APPROACH & KEY INSIGHTS

Solve using a 2D DP grid. If characters match, propagate the diagonal value. Otherwise, compute the minimum cost among the insert, delete, and replace choices + 1.

PERFECT JAVA SOLUTION

```
class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) dp[i][0] = i;
        for (int j = 0; j <= n; j++) dp[0][j] = j;
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1];
                else dp[i][j] = 1 + Math.min(dp[i - 1][j - 1], Math.min(dp[i - 1][j], dp[i][j - 1]));
            }
        }
        return dp[m][n];
    }
}
```

10. Backtracking

| 1. Subsets

PROBLEM DEFINITION

Given an integer array `nums` of **unique** elements, return **all possible subsets** (the power set)*. The solution set must not contain duplicate subsets. Return the solution in **any order**.

PRIMARY TEST CASES

```
Input: nums = [1,2,3]
Output: [[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]
```

APPROACH & KEY INSIGHTS

Standard backtracking framework template. Recursively step through the items, saving a deep copy of the active element list at each node.

PERFECT JAVA SOLUTION

```
class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(res, new ArrayList<>(), nums, 0);
        return res;
    }
    private void backtrack(List<List<Integer>> res, List<Integer> temp, int[] nums, int start) {
        res.add(new ArrayList<>(temp));
        for (int i = start; i < nums.length; i++) {
            temp.add(nums[i]);
            backtrack(res, temp, nums, i + 1);
            temp.remove(temp.size() - 1);
        }
    }
}
```

2. Permutations

PROBLEM DEFINITION

Given an array `nums` of distinct integers, return *all the possible permutations*. You can return the answer in **any order**.

PRIMARY TEST CASES

Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],...]

APPROACH & KEY INSIGHTS

Recursively build permutations by tracking used elements with a boolean array or `contains()` checks, removing the last element as you backtrack.

PERFECT JAVA SOLUTION

```
class Solution {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(res, new ArrayList<>(), nums);
        return res;
    }
    private void backtrack(List<List<Integer>> res, List<Integer> temp, int[] nums) {
        if (temp.size() == nums.length) { res.add(new ArrayList<>(temp)); return; }
        for (int i = 0; i < nums.length; i++) {
            if (temp.contains(nums[i])) continue;
            temp.add(nums[i]);
            backtrack(res, temp, nums);
            temp.remove(temp.size() - 1);
        }
    }
}
```

3. Combination Sum

PROBLEM DEFINITION

Given an array of **distinct** integers `candidates` and a target integer `target`, return **a list of all unique combinations** of `candidates` where the chosen numbers sum to `target`.

PRIMARY TEST CASES

Input: candidates = [2,3,6,7], target = 7
Output: [[2,2,3],[7]]

APPROACH & KEY INSIGHTS

Recursively accumulate candidates to match the target value. Since elements can be reused, pass the current index `i` forward instead of `i + 1`.

PERFECT JAVA SOLUTION

```
class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(res, new ArrayList<>(), candidates, target, 0);
        return res;
    }
    private void backtrack(List<List<Integer>> res, List<Integer> temp, int[] cand, int remain, int start) {
        if (remain < 0) return;
        if (remain == 0) { res.add(new ArrayList<>(temp)); return; }
        for (int i = start; i < cand.length; i++) {
            temp.add(cand[i]);
            backtrack(res, temp, cand, remain - cand[i], i);
            temp.remove(temp.size() - 1);
        }
    }
}
```

4. Generate Parentheses

PROBLEM DEFINITION

Given n pairs of parentheses, write a function to *generate all combinations of well-formed parentheses*.

PRIMARY TEST CASES

Input: $n = 3$

Output: ["((()))", "(()())", "(())()", "()(())", "()()()"]

APPROACH & KEY INSIGHTS

Track the counts of open and close brackets used. Add an open bracket if $\text{open} < n$, and add a closing bracket only if $\text{close} < \text{open}$.

PERFECT JAVA SOLUTION

```
class Solution {
    public List generateParenthesis(int n) {
        List res = new ArrayList<>();
        backtrack(res, new StringBuilder(), 0, 0, n);
        return res;
    }
    private void backtrack(List res, StringBuilder sb, int open, int close, int max) {
        if (sb.length() == max * 2) { res.add(sb.toString()); return; }
        if (open < max) {
            sb.append("("); backtrack(res, sb, open + 1, close, max);
            sb.setLength(sb.length() - 1);
        }
        if (close < open) {
            sb.append(")"); backtrack(res, sb, open, close + 1, max);
            sb.setLength(sb.length() - 1);
        }
    }
}
```

5. Word Search

PROBLEM DEFINITION

Given an $m \times n$ grid of characters `board` and a string `word`, return `true` if `word` exists in the grid*.

PRIMARY TEST CASES

Input: `board = [ ["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"] ]`, `word = "ABCCED"`
Output: `true`

APPROACH & KEY INSIGHTS

Perform a grid-based DFS exploration. Temporarily mask visited cells with a character placeholder like ``` to avoid reuse during the current search path.

PERFECT JAVA SOLUTION

```
class Solution {
    public boolean exist(char[][] board, String word) {
        for (int i = 0; i < board.length; i++) {
            for (int j = 0; j < board[0].length; j++) {
                if (dfs(board, word, i, j, 0)) return true;
            }
        }
        return false;
    }
    private boolean dfs(char[][] b, String w, int r, int c, int idx) {
        if (idx == w.length()) return true;
        if (r < 0 || c < 0 || r >= b.length || c >= b[0].length || b[r][c] !=
w.charAt(idx)) return false;
        char temp = b[r][c];
        b[r][c] = '`';
        boolean found = dfs(b, w, r + 1, c, idx + 1) || dfs(b, w, r - 1, c, idx + 1) ||
                        dfs(b, w, r, c + 1, idx + 1) || dfs(b, w, r, c - 1, idx + 1);
        b[r][c] = temp;
        return found;
    }
}
```

6. Palindrome Partitioning

PROBLEM DEFINITION

Given a string `s`, partition `s` such that every substring of the partition is a **palindrome**. Return **all** possible palindrome partitioning of `s`.

PRIMARY TEST CASES

Input: s = "aab"
Output: ["a", "a", "b"], ["aa", "b"]]

APPROACH & KEY INSIGHTS

Iterate through the string, checking if the current prefix substring is a palindrome. If valid, recursively backtrack on the remaining suffix.

PERFECT JAVA SOLUTION

```
class Solution {
    public List<String> partition(String s) {
        List<String> res = new ArrayList<>();
        backtrack(res, new ArrayList<>(), s, 0);
        return res;
    }
    private void backtrack(List<String> res, List<String> temp, String s, int start) {
        if (start == s.length()) { res.add(new ArrayList<>(temp)); return; }
        for (int i = start; i < s.length(); i++) {
            if (isPalindrome(s, start, i)) {
                temp.add(s.substring(start, i + 1));
                backtrack(res, temp, s, i + 1);
                temp.remove(temp.size() - 1);
            }
        }
    }
    private boolean isPalindrome(String s, int low, int high) {
        while (low < high) if (s.charAt(low++) != s.charAt(high--)) return false;
        return true;
    }
}
```

7. N-Queens

PROBLEM DEFINITION

The **n-queens** puzzle is the problem of placing n queens on an $n \times n$ chessboard such that no two queens attack each other.

PRIMARY TEST CASES

Input: $n = 4$
Output: Board configuration layout lists containing positions.

APPROACH & KEY INSIGHTS

Place queens row by row. Use boolean flag sets to quickly track which columns, positive diagonals, and negative diagonals are under attack.

PERFECT JAVA SOLUTION

```
class Solution {
    public List<List<String>> solveNQueens(int n) {
        List<List<String>> res = new ArrayList<>();
        char[][] board = new char[n][n];
        for (int i = 0; i < n; i++) Arrays.fill(board[i], '.');
        backtrack(res, board, 0, n);
        return res;
    }
    private void backtrack(List<List<String>> res, char[][] board, int row, int n) {
        if (row == n) { res.add(validateBoard(board)); return; }
        for (int col = 0; col < n; col++) {
            if (isValid(board, row, col, n)) {
                board[row][col] = 'Q';
                backtrack(res, board, row + 1, n);
                board[row][col] = '.';
            }
        }
    }
    private boolean isValid(char[][] board, int r, int c, int n) {
        for (int i = 0; i < r; i++) if (board[i][c] == 'Q') return false;
        for (int i = r - 1, j = c - 1; i >= 0 && j >= 0; i--, j--) if (board[i][j] == 'Q')
            return false;
        for (int i = r - 1, j = c + 1; i >= 0 && j < n; i--, j++) if (board[i][j] == 'Q')
            return false;
        return true;
    }
    private List<String> validateBoard(char[][] board) {
        List<String> list = new ArrayList<>();
        for (int i = 0; i < board.length; i++) list.add(new String(board[i]));
        return list;
    }
}
```